

Practical 12 – Echo and Chat Client-Server Using TCP/UDP Sockets

AIM

To implement echo and chat client-server applications using TCP/UDP sockets.

PART A – ECHO CLIENT-SERVER

Algorithm:

1. Create a server socket and bind it to an address and port.
2. Listen for and accept a client connection when using TCP.
3. The client sends a message to the server.
4. The server receives the message and sends the same message back.
5. The client receives and displays the echoed message.

PART A – INPUT / OUTPUT

Input: A message entered by the client.

Output: The same message is returned by the server and displayed by the client.

PART B – CHAT CLIENT-SERVER

Algorithm:

1. Create and configure server and client sockets.
2. Establish a connection for TCP, or use sendto/recvfrom for UDP.
3. Allow the client and server to send and receive messages alternately.
4. Display received messages on each side.
5. Stop communication when either side enters "exit".

PART B – INPUT / OUTPUT

Input: Messages entered by the client and server.

Output: Messages are exchanged and displayed at both ends until communication is terminated.

PROCEDURE

1. Save the client and server programs in separate Python files, or use a combined program that lets the user choose the role.
2. Run the server first.
3. Run the client in another terminal on the same machine or a reachable host.
4. Enter a message and observe the echo/chat response.
5. Enter "exit" to terminate the chat.

OBSERVATION

The echo server returned the message sent by the client. In the chat application, messages were exchanged between the client and server using socket communication.

RESULT

Echo and chat client-server applications were implemented and tested using socket programming.